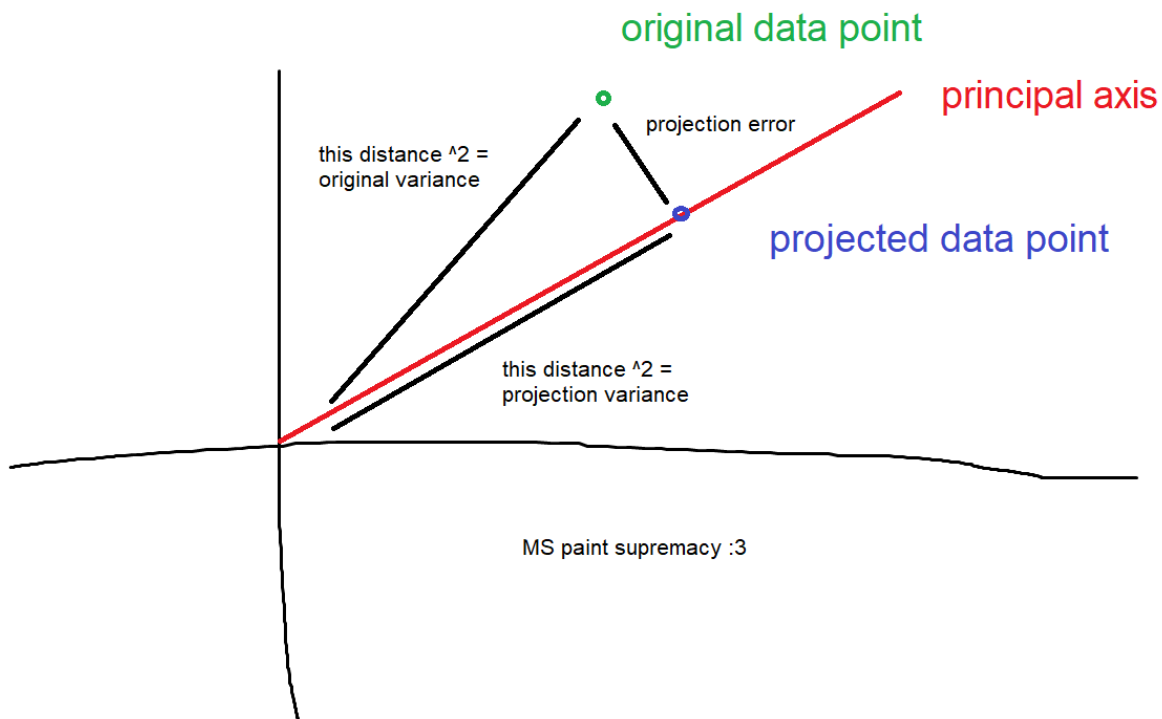


CSE 329 Term final solve 18-19 (Jan 20) Part B

5a/

5. (a) "Maximizing the projection variance on a principal axis is equivalent to minimizing the errors due to projection"-Justify the clause in the context of Principal Component Analysis (PCA). (10)

When projecting a data point from a higher dimension to a lower dimension, the information lost is the **projection error**. The original variance of a data point with respect to the origin is $\|x - 0\|^2 = \|x\|^2$. The **projection variance** is the variance of a data point with respect to the origin once it has been projected onto the principal / project axis. To visualize:



Applying pythagoras theorem:

$$\|x\|^2 = \text{projection variance} + \text{projection error}^2$$

Since the absolute or original variance is constant, an increase in projection variance results in a decrease in projection error. Principal Component Analysis results in mapping data points to vectors that results in the maximum projection variance, which in turn leads to the least amount of projection error / information loss incurred.

5b/

(b) What is eigenvector and what is eigenvalue? How are they related to the PCA? How does covariance matrix help to determine the principal components? Briefly explain. **(20)**

An eigenvector of a matrix is a special non-zero vector that does not change direction during matrix transformations, it only gets scaled. The amount by which it gets scaled is called the eigenvalue.

In the context of PCA, the eigenvector defines the principal axis / component, and its corresponding eigenvalue describes the amount of variance in the data explained by that principal component. A larger eigenvalue means higher spread / projection variance.

A ($p \times p$) covariance matrix with diagonal entries containing the variance of each individual feature and off diagonal entries containing the covariance between pairs of features captures the overall spread and correlation structure of features in the original data. Since the matrix is real and symmetric, it guarantees p real eigenvalues and p mutually orthogonal eigenvectors, which together help identify the principal components for PCA.

6a/

6. (a) How does Boosting help to get a better classifier? Briefly explain.

(10)

Boosting is a sequential ensemble method that builds a strong classifier by combining many weak learners. Rather than training each learner independently, boosting trains experts sequentially where each new learner focuses on the mistakes made by the previous ones. Boosting tries to reduce error by making each new model a "specialist" in the mistakes of its predecessors. Because each new model is forced to focus on the data points the previous models struggled with, this form of ensembling reduces bias by introducing more complexity with each new model. For the final output, models that are more accurate are given a higher **"Amount of Say"** (alpha), which leads to better predictions.

6b/ Nai?

7a/

7. (a) What is cost function? What problem does gradient descent face for a non-convex cost function? **(5+5)**

The cost function is a function $f(y, y')$ that enables the measurement of how well a model is performing. The value of the cost function increases as the model makes more mistakes and vice versa. The goal of training is to optimize a model's parameters such that the cost function $f(y, y')$ is minimized.

For cost functions that are not convex, the shape of the curve may not allow gradient descent to find the global minimum.

- Gradient descent can get stuck at local minimas in valleys generated by the cost function.

- Gradient descent can get stuck at regions called "Saddle Points" where the gradient is zero but it is neither minimum nor maximum.

- Flat regions or plateaus can slow down learning due to extremely small or zero gradient.

7b/

(b) Write down the main problem for each of the followings:

(4 x 2 $\frac{1}{2}$)

- i. Too simple hypothesis
- ii. Too complex hypothesis
- iii. Batch gradient descent
- iv. Stochastic gradient descent

i) Too simple hypothesis would not only lead to high testing error, but also high training error as the model is “too simple” to capture the intricacies of the dataset. This would lead to high bias.

ii) Too complex hypothesis would lead to extremely small training error but high testing error as the model captures not only the features of the training data but also the noise. This would lead to high variance and overfitting.

iii) Batch gradient descent involves updating weights after every epoch over the entire dataset. This is very memory intensive and extremely slow.

iv) Stochastic gradient descent involves updating weights after every sample is processed. This results in very noisy weight updates and leads to convergence issues.

7c/

(c) For which type of datasets it is useful to apply k -fold cross validation? Why may it work better than traditional 2-split (train and validation/test) approach in these datasets? Briefly explain. (10)

K-fold cross validation is useful for datasets where the amount of data is small and it is difficult to acquire new data.

In these situations keeping a separate validation set would result in depriving the model from valuable and scarce training data. The model

would end up depending heavily on samples that happened to be in the training set. This problem is even worse when the distribution of the types of samples between the training set and validation set are not the same, or when certain rare cases only appear in the validation set, which would result in the model always performing poorly. K-fold cross validation splits the data set into k-folds. The model trains on k-1 folds and validates on the remaining fold. This is repeated K times, each time with a different validation set / fold. The best performing model is selected, and trained on the entire dataset. This ensures that we select the best model and use the entirety of the dataset.

8a/

8. (a) How do we get sigmoid function from *logit* function? Why is sigmoid function important in logistic regression? **(5+5)**

$$h_{\theta}(x) = \theta^T x = \log \left(\frac{p}{1-p} \right)$$

$$e^{\theta^T x} = \frac{p}{1-p}$$

$$e^{\theta^T x} - e^{\theta^T x} p = p$$

$$e^{\theta^T x} = p(1 + e^{\theta^T x})$$

$$p = \frac{e^{\theta^T x}}{1 + e^{\theta^T x}}$$

$$p = \frac{1}{1 + e^{-\theta^T x}}$$

Logistic regression calculates a weighted sum of the input features just like linear regression. The problem is that this output can be any number between -inf to +inf, which we can not use for probability / classification. The sigmoid function clamps all values between -inf to +inf between 0 and

1, and is monotonically increasing, making it suitable for probability and hence classification. The sigmoid function is also continuous and differentiable, making it ideal for gradient descent and backpropagation.

$$s'(z) = s(z)(1 - s(z))$$

8bc/ Nai? Not sure